

앱 화면 · 게임 흐름

담당: mungim5556-ui (추천) · 생성형 AI 로 각 라운드를 진행하기 위한 안내서

이 문서로 할 수 있는 것

- ① 내가 맡은 작업과 순서를 안다
- ② 각 작업의 완료 기준을 안다
- ③ 내가 정해야 할 것과 팀과 맞춰야 할 것을 구분한다
- ④ AI 에게 어떤 순서로 무엇을 말할지 안다

내 라운드 한눈에 보기

라운드	작업 카드	비고
R0	B-0 통합 확인 · 폰 빌드	가장 먼저
R1	B-1 족지 좋아요 · 댓글 화면	필수
R1	B-2 방문한 장소의 족지만 열람	필수
R1	B-3 걷기 보상	기획 핵심
R2	B-4 AI 추천 루트 · 장소 공지 화면	필수
R3	B-5 퀘스트 탭 · 랜드마크 사진	게임성
R4	시연 시나리오 · 기획서 6번(화면) (공동)	제출

내 파일 (파일 주인)

UI/ , GameHUD.cs , CatchCameraScreen.cs , ArCatchRig.cs , PartnerBuddy.cs , Memo/ (앱 쪽),
GameProgress.cs , CollectController.cs , ItemCatalog.cs , Character/ , SampleScene.unity

다른 사람 파일을 고쳐야 하면 먼저 알리고, 그 사람을 PR 리뷰어로 지정합니다.

결정 노란 칸 = 작업자가 정할 것 (일부러 비워 둠) **연결** 파란 표시 = 다른 팀원과 먼저 합의할 것

시작 전 준비

1. 저장소 — 팀 원본 저장소 `mungim5556-ui/KUCA` 를 기준으로 작업합니다. 이 가이드의 원본 문서는 저장소의 `docs/guides/` 폴더에 있습니다 (main 에 합쳐진 뒤).
2. AI 도구 — 저장소 파일을 직접 읽고 고칠 수 있는 도구를 권장합니다 (예: Claude Code 를 GitHub 저장소에 연결). 대화형 AI 만 쓴다면 관련 파일 내용을 복사해 함께 붙여 넣습니다.
3. 라운드 순서 — R0 통합 → R1 필수 기능 → R2 AI 핵심 → R3 게임성 → R4 제출 준비. R1 과 R2 는 세 사람이 동시에 진행합니다.
4. 이 문서에서 비어 있는 칸 — **결정** 표시가 붙은 칸은 일부러 비워 두었습니다. 작업자가 정하고, 저장소 문서의 같은 칸에 **결정: 내용 — 이름, 날짜, 이유** 로 적어 PR 에 함께 넣습니다.

1부. 공통 — 생성형 AI 로 작업하는 방법

역할별 가이드(`A_아트_출현규칙.md` , `B_애플리케이션.md` , `C_서버_AI.md`)를 쓰기 전에 이 문서를 먼저 읽습니다. 전체 라운드와 역할은 `../로드맵_역할분담.md` 에 있습니다.

이 가이드를 읽는 법

각 작업 카드에는 다섯 칸이 있습니다.

칸	뜻
목표 · 완료 기준	팀이 합의한 것. 바꾸려면 팀에 먼저 말합니다
지금 코드	출발점이 되는 파일. AI에게 그대로 알려 주세요
결정 작업자가 결정할 것	일부러 비워 둔 칸. 작업자가 정하고 결정: 뒤에 적습니다
연결 맞춰야 할 것	다른 사람 작업과 이어지는 약속. 정하기 전에 상대와 합의합니다
AI 진행 순서	AI에게 시킬 순서와 예시 문장

결정 칸은 아래처럼 채우고 같은 PR에 넣습니다. 나중에 AI가 이 기록을 읽고 일관되게 작업합니다.

- 몇 개의 시간대로 나눌지 결정: 4개 (아침/낮/저녁/밤) — 최원준, 10/10. 이유: 수업 시간표와 맞추기 쉬움

작업 한 건의 흐름

1. 카드 고르기 — 자기 역할 가이드에서 이번 라운드 카드를 고릅니다.
2. 브랜치 — 최신 main에서 `feature/<이름>-<기능>` 브랜치를 만듭니다.
3. 결정 먼저 — **결정** 칸을 AI와 상의해 정하고 문서에 적습니다. **연결** 칸은 상대방에게 확인받습니다.
4. 설계 → 구현 — AI에게 바로 코드를 쓰게 하지 말고, 먼저 계획을 받고 확인한 뒤 진행합니다.
5. 확인 — 카드의 "확인 방법"을 직접 해 봅니다. Unity는 에디터 Play, 가능하면 폰.
6. PR — main으로 PR을 열고, 파일 주인이 다른 파일을 건드렸다면 그 사람을 리뷰어로 지정합니다.

AI에게 줄 문장 (복사해서 쓰기)

세션 시작

이 저장소는 경희대 국제캠퍼스 산책·수집 게임 KUCA입니다 (Unity 6 URP 앱 + `server/` ASP.NET 메모 서버, Azure 배포). `docs/로드맵_역할분담.md`, `docs/guides/00_공통_AI작업가이드.md`, `docs/guides/〈내 역할〉.md` 를 먼저 읽으세요. 이번 작업은 카드 "<카드 번호와 이름>" 입니다. 아직 코드는 쓰지 말고, 관련 파일을 읽은 뒤 (1) 지금 구조 요약 (2) **결정** 결정할 것마다 선택지 2~3개와 장단점 (3) 구현 계획 을 알려 주세요.

결정 후 구현

결정은 이렇게 했습니다: <결정 내용>. 가이드의 해당 **결정** 칸에 기록하고, 계획대로 구현하세요. 내 파일 주인 범위 밖의 파일은 꼭 필요할 때만 최소로 고치고, 고친 이유를 알려 주세요. 끝나면 내가 직접 확인할 방법(눌러 볼 순서)을 알려 주세요.

리뷰 받기

main 과 비교한 이번 브랜치 변경을 검토해 주세요. 버그, 다른 팀원 파일과 충돌할 곳, 빠진 예외 처리(네트워크 실패, 위치 권한 없음 등)를 찾아 주세요.

막혔을 때

<증상>이 생깁니다. 재현 순서: <순서>. 에러 로그: <로그>. 추측으로 고치지 말고 원인부터 찾아 주세요.

AI에게 맡길 때 지킬 것

- 한 번에 한 카드. 여러 기능을 한 PR에 섞지 않습니다.
- Unity 씬(`SampleScene.unity`)은 AI가 직접 고치지 않게 합니다. 오브젝트는 코드에서 만들거나, 씬 수정 이 필요하면 사람이 에디터에서 합니다.
- `.meta` 파일은 Unity가 만든 것을 그대로 커밋합니다. AI가 임의로 만들면 GUID가 꼬입니다.
- API 키·비밀번호는 저장소에 넣지 않습니다. 서버 키는 Azure 앱 설정 / 로컬은 `dotnet user-secrets` . 앱 (Unity)에는 AI 키를 넣지 않고 항상 서버를 거칩니다.
- AI가 "됐다"고 해도 직접 돌려 봅니다. 이 프로젝트는 AI가 Unity를 실행해 볼 수 없는 환경이 많습니다.
- 기획서와 다른 판단을 하게 되면 PR 설명에 적습니다 (발표 때 기획서와 맞춰야 하므로).

역할 사이 약속 (**연결** **모음**)

여기 있는 약속은 정해지면 이 표에 적습니다. 한쪽이 바꾸면 다른 쪽이 깨지는 것들입니다.

약속	주는 쪽 → 쓰는 쪽	정할 것	결정
출현 조건 데이터	A → B (도감 "언제 나와요" 힌트)	파일 위치·형식	
희귀 색 개체 표시	A → B (<code>GameProgress.Caught</code> 필드)	필드 이름·값	
좋아요·댓글 API	C → B	<code>docs/api.md</code> 에 추가	
쪽지 필터 거절 응답	C → B	상태 코드·사용자에게 보일 문구	
탐방 추천 API	C → B	입력·출력 JSON	
공지 API	C → B	공지 ↔ 장소 ID 연결 형식	
날씨 정보	C 또는 A → A	앱에서 직접 vs 서버 경유	
퀘스트 보상	A(액세서리) → B(지급)	아이템 id 목록	

API는 항상 `docs/api.md` 를 먼저 고친 PR로 합의하고, 그다음 서버·앱을 구현합니다.

2부. 팀원 B 작업 카드

파일 주인: UI/ , GameHUD.cs , CatchCameraScreen.cs , ArCatchRig.cs , PartnerBuddy.cs , Memo/ (앱 쪽), GameProgress.cs , CollectController.cs , ItemCatalog.cs , Character/ , SampleScene.unity

먼저 읽기: 00_공통_AI작업가이드.md , ../api.md

지금 상태 요약

- 메뉴: `KhuMenus` 에 메인 메뉴 · 도감 · 상세 · 스크랩북 · 프로필이 있습니다. 퀘스트 탭은 없습니다.
- 잡기: 지도에서 대상 탭 → `CatchCameraScreen` 에서 사진을 찍어야 잡힘 → `CollectController.CompleteCatch` → `GameProgress` .
- 쪽지: `MemoBoard` 가 `IMemoService` (`GetMemos` , `CreateMemo` , `DeleteMemo` , `LoadPhoto`)로 서버와 통신. 작성만 스팟 근처로 제한되고, 읽기는 어디서나 됩니다. 좋아요·댓글 없음.
- 걸은 거리는 `progress.walkedMeters` 로 기록만 합니다.
- 공지: `Data/Notices.json` 을 그대로 보여 줍니다.

B-0. 통합 확인 · 폰 빌드 (R0, 가장 먼저)

완료 기준: main 에서 Unity 컴파일 에러 0, 안드로이드 또는 iOS 기기에서 지도 → 잡기 → 도감 → 쪽지가 한 바퀴 돈다.

결정 작업자가 결정할 것

- 시연 기기 (안드로이드 / iOS / 둘 다)

결정 :

- 발견된 문제를 어디에 모을지 (GitHub Issues 권장)

결정 :

AI 진행 순서: 컴파일 에러가 나면 에러 로그 전체를 AI에게 주고 "원인 파일과 최소 수정"을 요청. 다른 사람 파일이면 그 사람에게 알리기.

B-1. 쪽지 좋아요 · 댓글 화면 (R1, 필수)

목표: 기획서 "다른 사용자도 그 쪽지를 볼 수 있고 좋아요나 댓글을 통해서 반응을 남길 수 있다."

완료 기준: 쪽지에 좋아요를 누르고 취소할 수 있고, 댓글을 쓰고 볼 수 있다. 서버 실패 시 안내가 뜬다.

지금 코드: `Memo/MemoBoard.cs` , `Memo/IMemoService.cs` , `Memo/HttpMemoService.cs` ,
`Memo/MockMemoService.cs`

결정 작업자가 결정할 것

- 댓글 길이 제한, 댓글에 사진 허용 여부

결정 :

- 댓글을 쪽지 상세 안에 펼칠지, 별도 화면으로 갈지

결정 :

- 좋아요를 누른 순간 화면을 먼저 바꿀지(빠름), 서버 응답 후 바꿀지(정확)

결정 :

- 내 쪽지에 반응이 오면 알려 줄지 (알림은 범위가 커짐)

결정 :

연결 맞춰야 할 것: 좋아요·댓글 API 모양 → C 가 `docs/api.md` 에 먼저 추가, 함께 확인. 서버가 준비되기 전에는 `MockMemoService` 에 같은 기능을 넣어 화면부터 진행.

AI 진행 순서: "IMemoService 에 좋아요·댓글 메서드를 추가하고 Mock 부터 구현해 주세요" → 화면 → C 의 서버가 나오면 `HttpMemoService` 연결.

B-2. 방문한 장소의 쪽지만 열람 (R1, 필수)

목표: 기획서 "그곳을 방문한 학생만 본다."

완료 기준: 한 번도 가 본 적 없는 스팟의 쪽지는 내용이 가려지고, 가 보면 열린다.

지금 코드: `MemoBoard.CanWrite` , `GameProgress.visitedSpots` , `GameProgress.VisitSpot`

결정 작업자가 결정할 것

- "방문"의 기준 (스팟 반경 안에 한 번 들어옴 / 몇 초 머무름)

결정 :

- 가려진 쪽지를 어떻게 보여 줄지 (개수만 / 흐린 미리보기 / 아예 숨김)

결정 :

- 앱에서만 막을지, 서버에서도 막을지 (서버에서 막으려면 C 와 방문 기록 공유 필요)

결정 :

B-3. 걷기 보상 (R1)

목표: 기획서 "걷기 보상으로 한 마리 더 포획할 수 있게."

완료 기준: 일정 거리를 걸으면 보상이 생기고, 사용자가 그걸 알 수 있다.

지금 코드: `KhuMenus.cs` (`walkedMeters` 증가), `ItemCatalog.cs`, `CollectibleSpawner.cs` (A 소유)

결정 작업자가 결정할 것

- 보상 형태 (근처에 특별 출현 / 아이템 지급 / 잡기 횟수)

결정 :

- 몇 m마다, 하루 상한을 둘지

결정 :

- GPS 튜브(가만히 있는데 거리 증가)을 어떻게 걸러낼지

결정 :

연결 맞춰야 할 것: 보상이 "특별 출현"이면 `CollectibleSpawner` 에 출현 요청 함수가 필요 → A 와 함수 모양 합의.

B-4. AI 추천 루트 · 장소 공지 화면 (R2, 필수)

목표: C 가 만든 AI 추천과 공지 판별 결과를 지도 위에 보여 줌.

완료 기준: 버튼 하나로 추천을 받고, 추천 장소들이 지도에 순서대로 표시된다. 장소를 누르면 그 장소 공지가 원문 링크와 함께 보인다. AI 실패 시에도 화면이 깨지지 않는다.

결정 작업자가 결정할 것

- 추천을 받을 때 사용자에게 무엇을 물을지 (남은 시간만 / 기분·취향도)

결정 :

- 추천 루트 표시 방식 (선 / 번호 핀 / 둘 다), 추천 이유 문장을 어디에 보일지

결정 :

- 응답을 기다리는 동안 보여 줄 화면

결정 :

- 공지를 장소 쪽지판 안에 넣을지, 따로 보일지

결정 :

B-5. 퀘스트 탭 · 랜드마크 사진 모아 보기 (R3)

목표: 기획서 "퀘스트 - 주변 유저들과 연계하여 활동하는 퀘스트가 제시되고, 클리어 시 보상", "랜드마크를 누르면 유저들이 업로드한 여러 사진 조회 가능, 가장 인기 많은 사진이 대표 사진."

완료 기준: 퀘스트 목록이 보이고, 조건을 채우면 보상을 받는다. 랜드마크에서 사진들을 모아 보고 대표 사진이 정해진다.

결정 작업자가 결정할 것

- 첫 버전 퀘스트 종류 (예: 처음 가는 랜드마크 방문, 특정 캐릭터 잡기, 쪽지 남기기, 신입생 온보딩 코스)

결정 :

- 퀘스트 갱신 주기 (일일 / 주간 / 고정)

결정 :

- 퀘스트를 앱에 고정할지, 서버에서 내려받을지 (운영 중 바꾸려면 서버)

결정 :

- 대표 사진 기준 (좋아요 수 / 최신 / 기획서 원문은 "가장 인기 많은" — 문구 확인 필요)

결정 :

연결 맞춰야 할 것: 보상 아이템 id → A, 서버형 퀘스트·사진 목록이면 → C

공용 데이터 주인으로서 할 일

GameProgress.cs 는 여러 기능이 기록을 남기는 파일입니다. A·C 가 필드를 추가해 달라고 하면 B 가 넣고, 예전 저장 파일이 깨지지 않게 Migrate() 에 기본값 처리를 추가합니다.